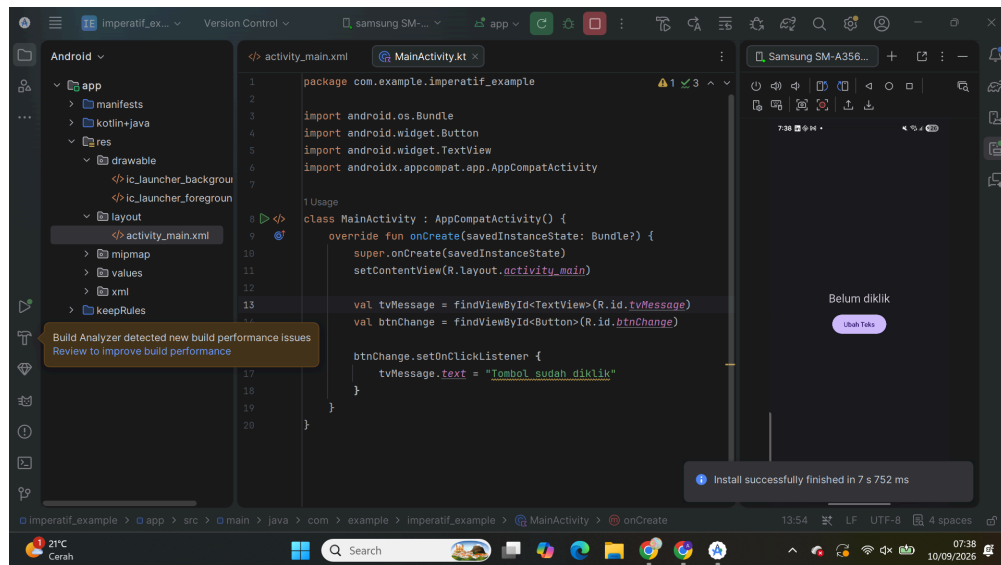


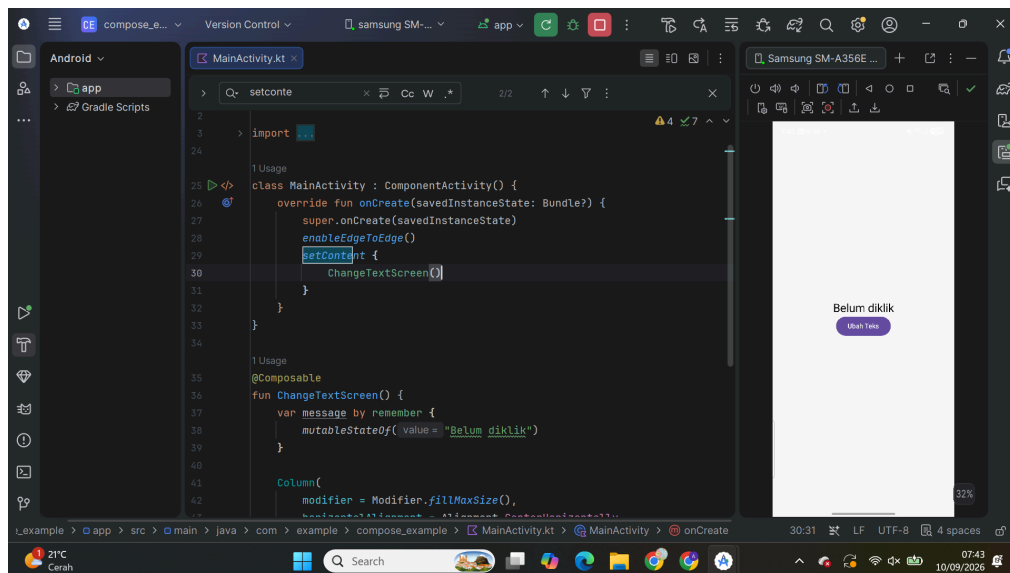
Nama: Andien Oktriarahmah Syarifah
NIM: 245150400111046
Mata Kuliah: Pengembangan Aplikasi Mobile
Kelas: F
MODUL 3 - Event, State, Imperatif

Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Menjalankan Project A dan B
2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.



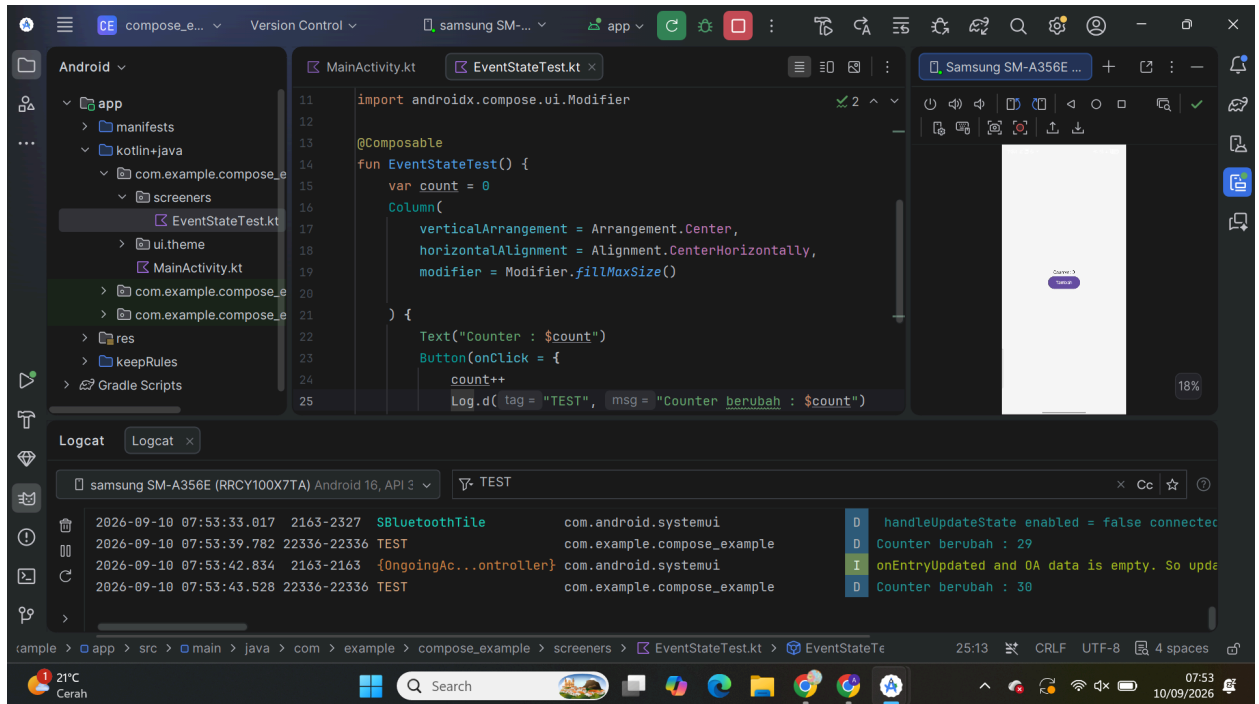
3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.



4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?
Karena Jetpack Compose menggunakan pendekatan deklaratif. Jadi tidak perlu mengubah objek teks (UI) secara langsung, melainkan cukup mengubah *state*-nya. Ketika *state* diubah, compose akan secara otomatis memperbarui UI yang bergantung pada *state* tersebut.
5. Jelaskan alur perubahan pada XML/View dengan pola: event → View → perubahan tampilan.
Alur perubahannya adalah: event → cari View (seperti `findViewById`) → ubah properti View secara langsung (contohnya mengubah nilai `text`).
6. Jelaskan alur perubahan pada Compose dengan pola: event → state → recomposition → perubahan tampilan.
Alur perubahannya adalah: event → ubah *state* → Compose secara otomatis mendeteksi perubahan tersebut dan memperbarui UI melalui proses *recomposition*.
7. Apa yang terjadi jika `message` diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.
Jika diganti dengan variabel biasa, perubahan pada teks tidak akan memperbarui UI. Karena compose hanya melakukan *recomposition* jika variabel tersebut merupakan sebuah *state* seperti `mutableStateOf` yang dapat diamati perubahannya. Variabel lokal biasa dapat berubah di memori, namun tidak otomatis meminta compose untuk memperbarui tampilan.
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.
Yang saya dapatkan adalah dengan cara berpikir imperatif seperti memberi perintah manual langkah demi langkah kita harus mencari elemen UI-nya lalu mengubah datanya secara langsung. Sedangkan jika cara berpikir deklaratif berfokus pada data (*state*) kita hanya merancang UI berdasarkan *state*, dan UI akan otomatis menyesuaikan dirinya sendiri setiap kali data *state* tersebut diubah.

Event Handling State

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
Counter-nya bertambah sesuai dengan berapa kali diklik. Di LogCat terlihat sudah 30 kali diklik dan counter sudah sampai 30.
3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
Tidak terjadi perubahan apa apa pada tampilan.
4. Apa yang menyebabkan hal tersebut terjadi?
Karena variabel count hanyalah variabel biasa, bukan merupakan state (seperti mutableStateOf). Perubahan nilai pada variabel biasa tidak akan memicu Compose untuk melakukan recomposition atau menggambar ulang tampilan.
5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.

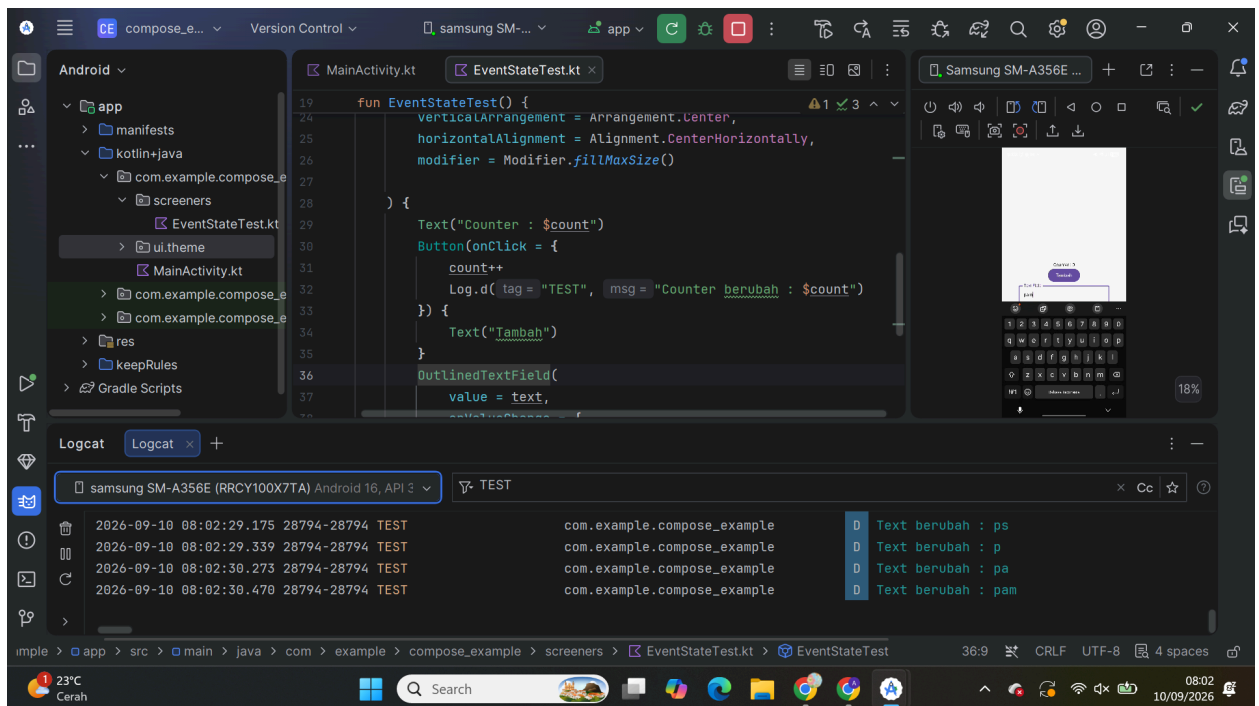
- Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

```
var count = 0
```

menjadi:

```
var count by remember { mutableStat
```

- Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



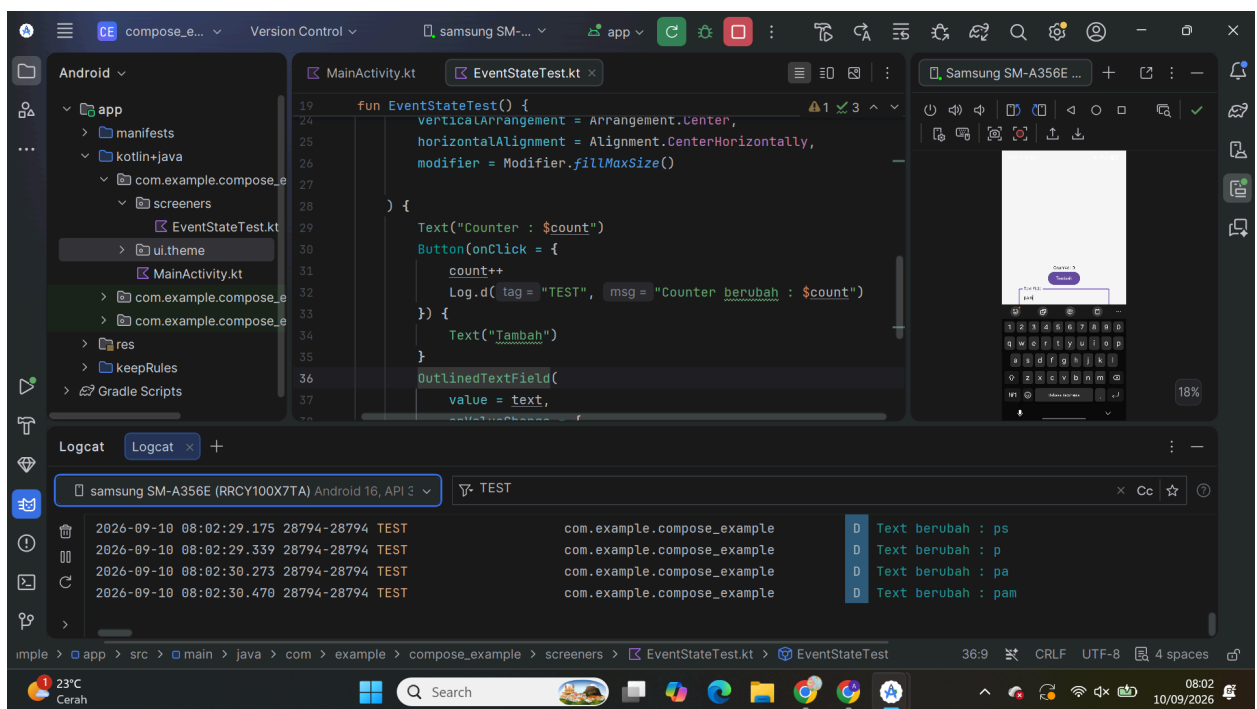
- Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
Counternya bertambah.
- Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
Teks pada tampilan aplikasi sekarang akan berubah dengan benar dan menampilkan angka yang bertambah sesuai dengan jumlah klik.
- Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?
Perbedaan terjadi karena variabel count kini menggunakan fungsi state yaitu mutableStateOf(), sehingga Compose dapat mengamati perubahan datanya. Ketika nilai state berubah, maka proses recomposition terjadi dan UI akan diperbarui. Selain itu, ada penggunaan fungsi remember yang berfungsi untuk menyimpan nilai state tersebut agar tidak ter-reset kembali ke nilai awalnya pada saat layar digambar ulang

11. Apa yang dimaksud recomposition pada Compose?

Proses ketika Compose memperbarui bagian atau komponen UI tertentu sebagai akibat dari adanya *state* (data) yang mengalami perubahan.

12. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:

13. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



14. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Tulisan "Text berubah : " yang diikuti oleh karakter atau string terbaru yang diketikkan.

15. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Kotak Text Field pada aplikasi akan menampilkan huruf dan kalimat yang sedang diketik oleh pengguna secara langsung.

16. Apa yang menyebabkan hal tersebut terjadi?

Aktivitas mengetik memicu event bernama `onValueChange` yang ada pada `OutlinedTextField`. Event ini kemudian memperbarui state dari variabel `text`. Karena `text`

adalah sebuah state, setiap perubahan nilainya memicu recomposition sehingga tampilan form di layar juga ikut diperbarui.

17. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Count dan text.

18. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

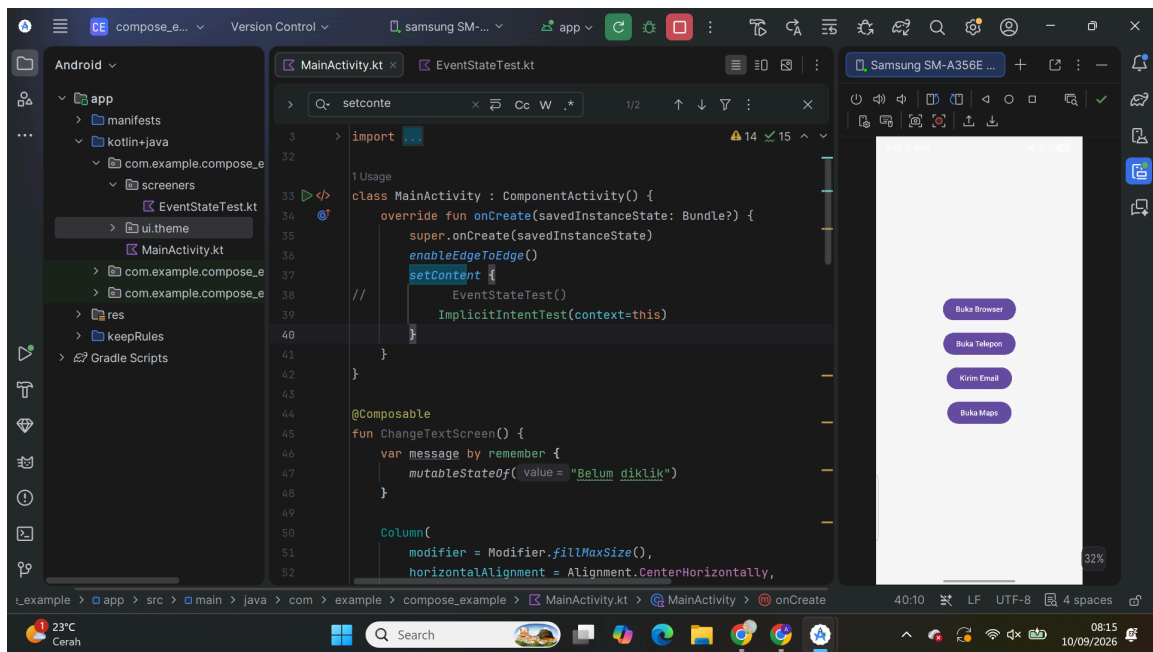
onClick dan onChange.

19. Kapan event - event tersebut dijalankan?

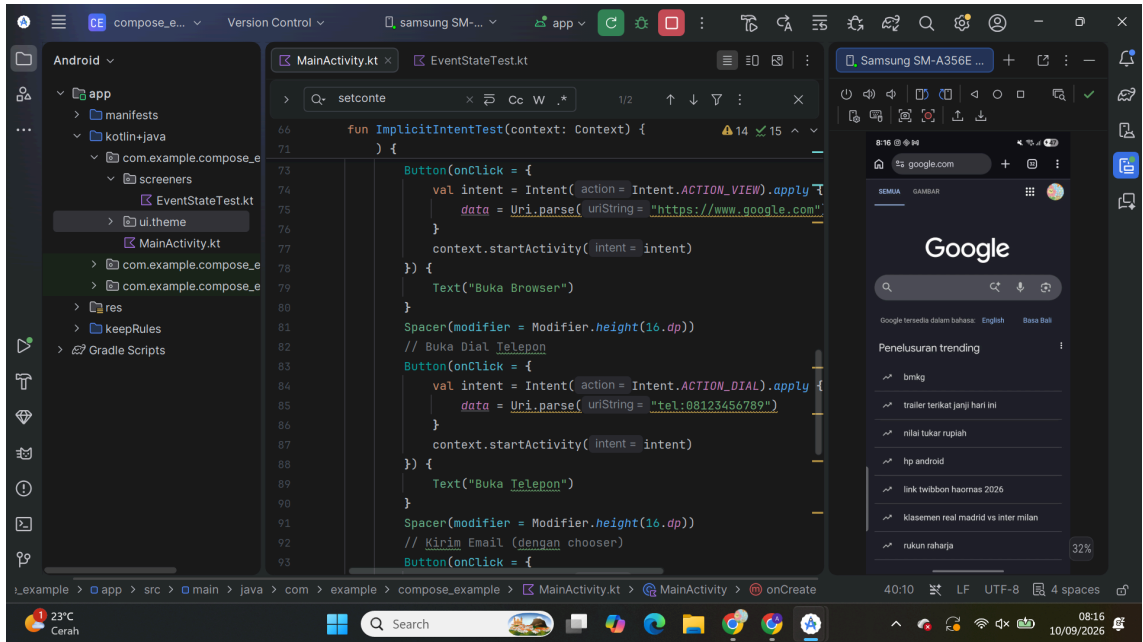
Event onClick dijalankan ketika pengguna melakukan interaksi mengeklik tombol Button, sedangkan event onChange dijalankan setiap kali pengguna mengetik, menghapus, atau mengubah teks yang ada di dalam kotak OutlinedTextField

Implisit Intent

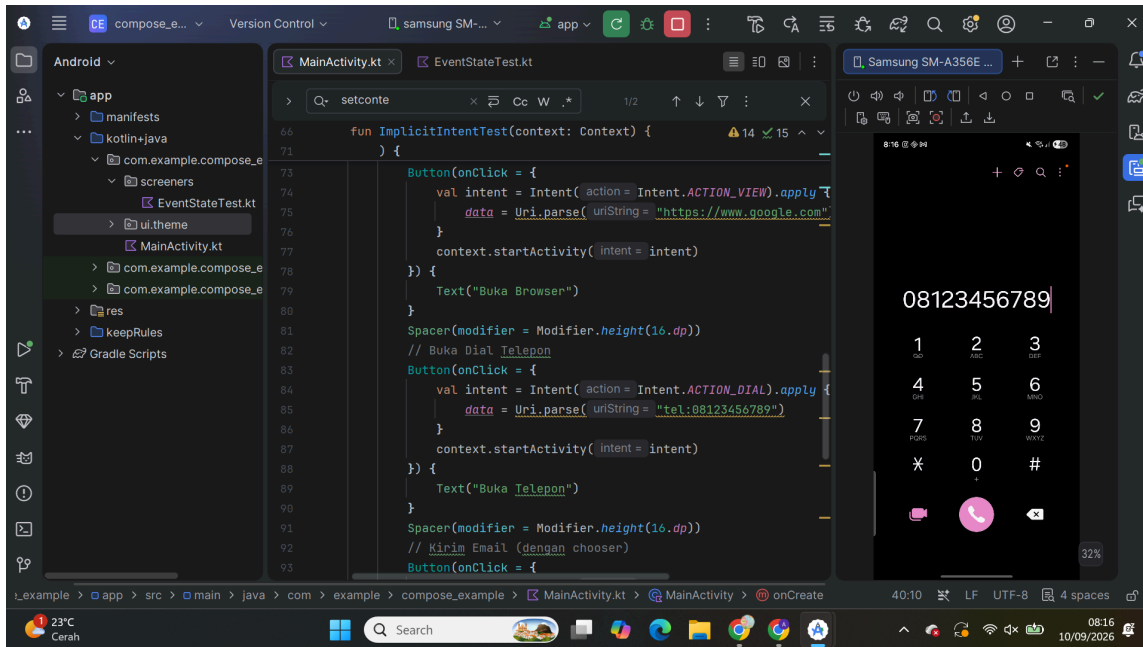
1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol.



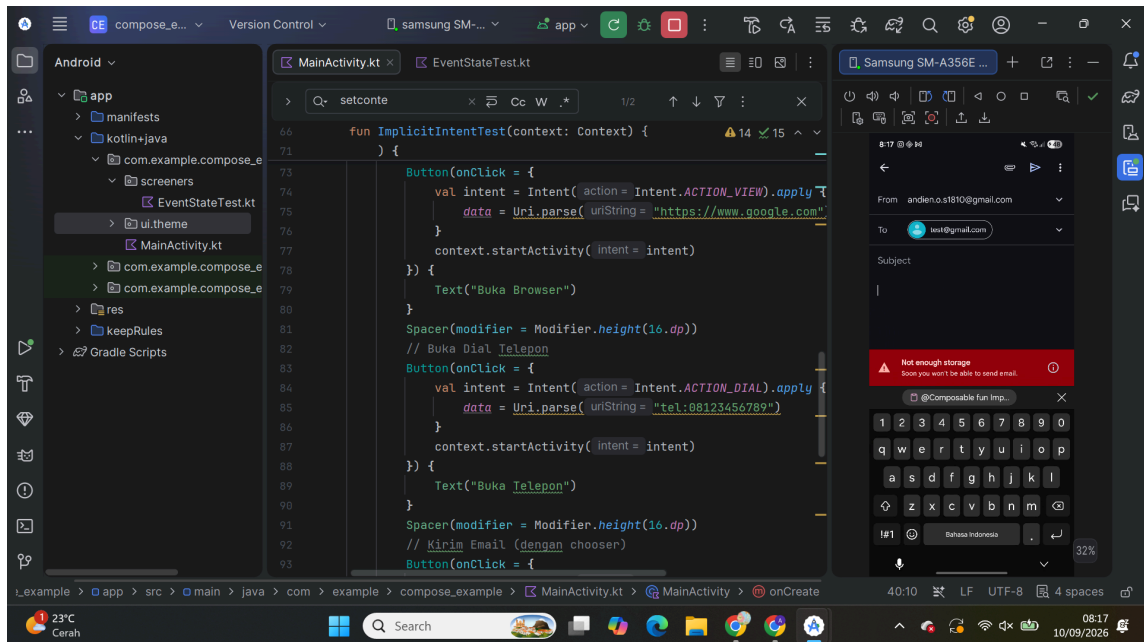
2. Klik Buka Browser



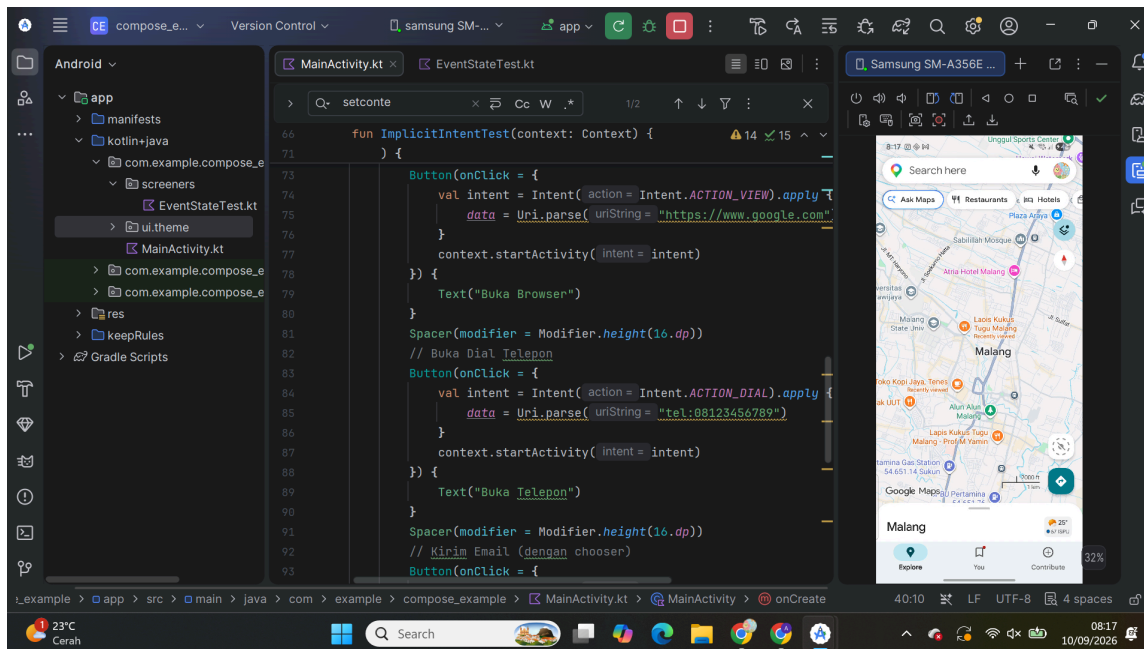
3. Klik Buka Telepon



4. Klik Kirim Email



5. Klik Buka Maps



6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya
7. Pada kode yang digunakan, terdapat beberapa action seperti:
8. Intent.ACTION_VIEW
9. Intent.ACTION_DIAL
10. Intent.ACTION_SENDTO
11. Apa perbedaan fungsi dari masing-masing action intent tersebut?

- Intent.ACTION_VIEW: Digunakan secara umum untuk menampilkan suatu data atau memanggil aplikasi yang sesuai untuk membuka data tersebut, misalnya membuka URL web di peramban (browser) atau membuka koordinat lokasi.
- Intent.ACTION_DIAL: Digunakan khusus untuk membuka aplikasi telepon (dialer) bawaan dengan nomor telepon yang sudah disiapkan tanpa langsung memanggil/menelepon.
- Intent.ACTION_SENDTO: Digunakan khusus untuk mengirimkan suatu pesan ke penerima yang spesifik, seperti membuka aplikasi email atau pengolah pesan dengan alamat tujuan yang telah diisi.

12. Pada kode yang digunakan, terdapat beberapa data seperti:

13. data = Uri.parse("mailto:test@gmail.com")

14. data = Uri.parse("<https://www.google.com>")

15. data = Uri.parse("geo:-7.250445,112.768845")

16. Apa fungsi data pada intent ?

Data berfungsi untuk memberikan nilai atau informasi spesifik yang diperlukan oleh action, seperti format URL (untuk browser), nomor telepon (tel:), alamat email (mailto:), atau format koordinat (geo:) yang akan dieksekusi.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Action mendefinisikan "apa yang ingin dilakukan", sedangkan *Data* memberitahu sistem "apa yang akan diproses". Pada konsep *Implicit Intent*, sistem operasi Android membaca kombinasi *Action* dan format URI pada *Data* tersebut untuk kemudian memilihkan dan menawarkan aplikasi mana yang paling cocok terinstall di perangkat untuk menjalankan perintah pengguna (misal: action SENDTO dengan data mailto akan otomatis membuka aplikasi email).